UNIVERSIDAD PERUANA UNIÓN

Facultad de Ingeniería y Arquitectura
Escuela Profesional de Ingeniería de Sistemas

INFORME DE ACTIVIDAD AUTÓNOMA · SESIÓN 11

Persistencia local y arquitectura Offline-First en PharmaMobil

SQLDelight 2.1.0 · SQLite · sincronización de productos · Kotlin Multiplatform

Dato académico	Información
Asignatura	Desarrollo de Aplicaciones Móviles
Estudiante	Andrey Fidel Mestanza Bazán
Docente	Mg. Benjamín David Reyna Barreto
Ciclo y semestre	VI ciclo · 2026-II
Fecha	21 de octubre de 2026

Lima, Perú · 2026

1. Introducción y objetivos

Una aplicación farmacéutica necesita conservar la consulta del inventario y permitir registrar productos cuando la conexión no está disponible. En PharmaMobil, SQLDelight convierte SQLite en la fuente local observada por la interfaz. El repositorio envía los registros pendientes a PharmaSoft cuando se restablece la red y actualiza después el catálogo local.

El objetivo de esta actividad fue demostrar: (1) un esquema local para productos y estado pendiente; (2) consulta reactiva desde la base; (3) altas sin red con identificadores negativos; (4) reconciliación local transaccional con el identificador del servidor; y (5) conservación del inventario ante un error de conexión.

Alcance de la validación

La lógica reside principalmente en **shared/commonMain**. Hay creadores de driver Android e iOS, pero las pruebas ejecutadas y capturas de esta entrega son de Android; no se afirma haber compilado ni probado iOS. La verificación usa un emulador Android, pruebas de host con SQLite en memoria y MockEngine, un archivo SQLite exportado y una consulta autenticada al backend local.

2. Diseño e implementación

El esquema **Producto.sq** define ocho columnas: id, nombre, precio, stock, activo, imagen, categoria y pendiente. Las consultas **listarTodos** y **listarPendientes** alimentan la lectura; **insertar**, **borrarPorId**, **borrarSincronizados** y **siguienteIdLocal** sostienen las operaciones de persistencia. No existe una consulta SQL llamada reemplazarIdTemporal.

Componente	Papel en el flujo
ProductoLocal	Convierte las consultas SQLDelight en Flow, asigna IDs negativos y ejecuta la sustitución del ID dentro de una transacción SQLite.
ProductoRepository	Contrato de dominio para observar, registrar, sincronizar y consultar la última hora de sincronización.
ProductoRepositorioOffline	Coordina SQLite y Ktor. Un Mutex compartido serializa registro y sincronización para impedir publicaciones concurrentes duplicadas.
ProductoViewModel	Presenta el Flow local, el aviso de conectividad y el estado de productos pendientes.

El alta sin red guarda primero el producto con ID negativo y pendiente = 1. La sincronización publica los pendientes, recibe un ID positivo y, en una transacción local, borra la fila temporal e inserta la respuesta remota con pendiente = 0. Después descarga los productos del servidor sin borrar los pendientes que todavía no se hayan enviado. El Mutex evita que un alta simultánea con una sincronización publique dos veces el mismo registro.

3. Evidencias de ejecución en Android

Las capturas siguientes corresponden al recorrido visible en el emulador. Cada una documenta el estado de la interfaz, no una medición de tiempo ni una prueba de reinicio completo de la aplicación.

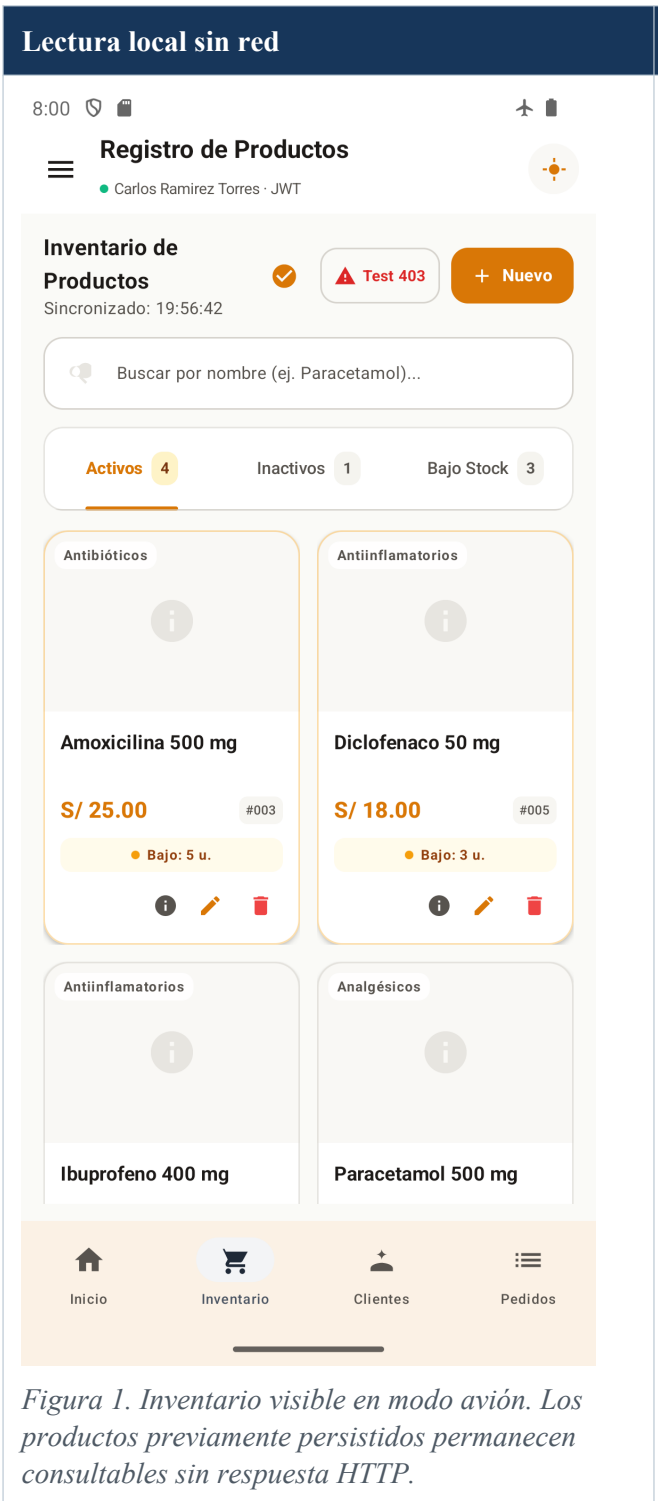


Figura 1. Inventario visible en modo avión. Los productos previamente persistidos permanecen consultables sin respuesta HTTP.

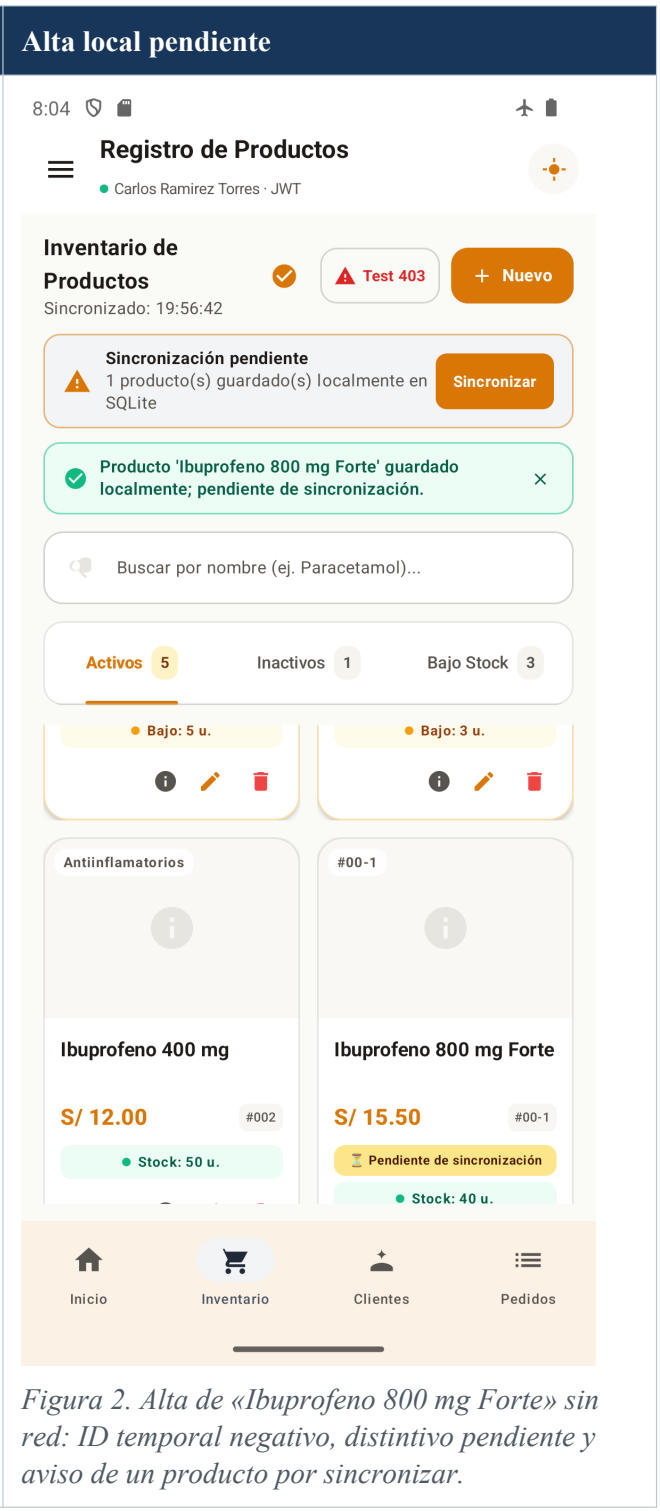


Figura 2. Alta de «Ibuprofeno 800 mg Forte» sin red: ID temporal negativo, distintivo pendiente y aviso de un producto por sincronizar.

3. Evidencias de ejecución en Android (continuación)

Reconciliación visible

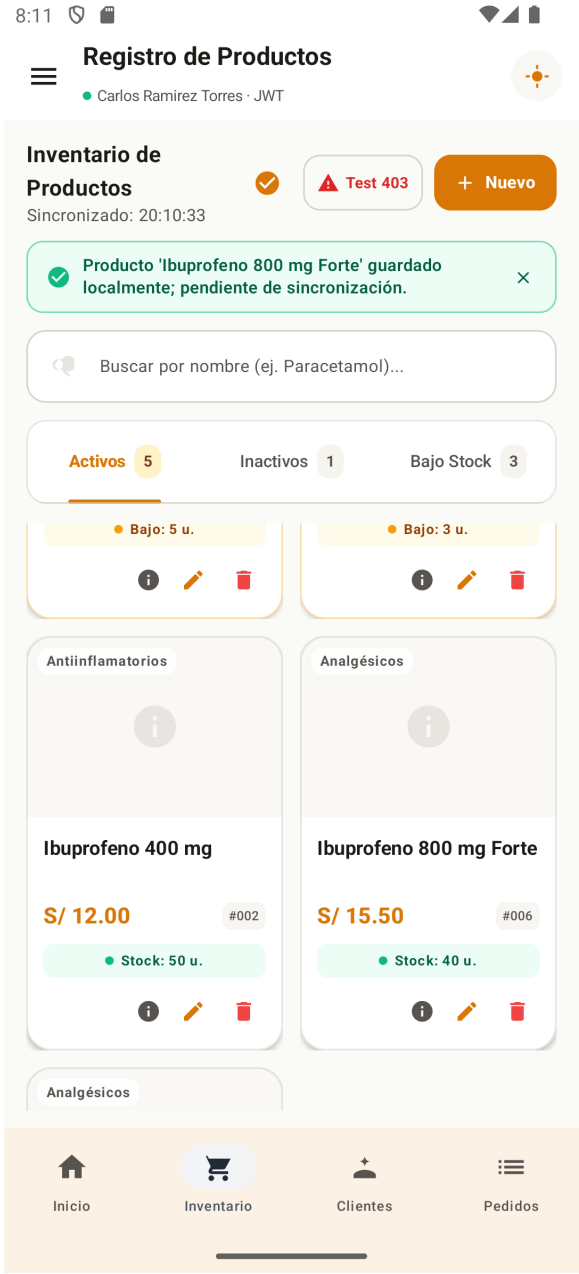


Figura 3. Tras reconectar, el producto aparece con ID #006 y sin distintivo pendiente. El mensaje transitorio de alta local aún visible no prueba por sí solo la sincronización.

Resiliencia ante error

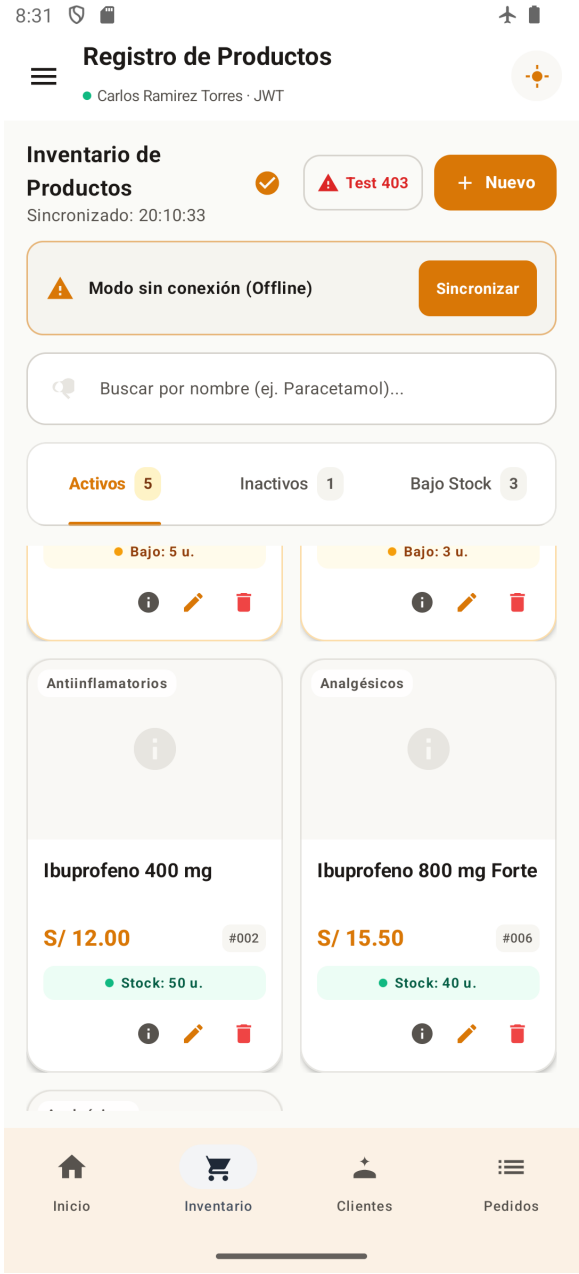


Figura 4. El aviso «Modo sin conexión» convive con el catálogo local; un fallo de red no vacía la lista visible.

La captura anteriormente rotulada como «Database Inspector» se excluye como evidencia del IDE porque su procedencia no pudo verificarse. Se reemplaza por una consulta reproducible del archivo SQLite exportado en la sección siguiente.

4. Verificaciones reproducibles

4.1 Base SQLite exportada

El archivo de evidencia **pharma_synced.db** se abrió como SQLite. **PRAGMA table_info(producto)** devuelve las ocho columnas esperadas; **SELECT COUNT(*), SUM(pendiente) FROM producto** devuelve 6 productos y 0 pendientes. El registro ID 6 contiene «Ibuprofeno 800 mg Forte», precio S/ 15.50 y stock 40. Esta inspección acredita el archivo exportado, pero no sustituye una captura auténtica del Database Inspector de Android Studio.

4.2 Backend PharmaSoft

Una solicitud autenticada a **GET /api/v1/productos?pagina=0&tamanio=100** devolvió seis productos; el ID 6 coincide en nombre, precio y stock con la base exportada. La misma ruta sin JWT respondió **401**. La verificación se hizo por cliente HTTP; no se presenta como captura de la respuesta en Swagger UI.

Fuente	ID	Producto	Precio	Stock	Estado
SQLite exportado	6	Ibuprofeno 800 mg Forte	S/ 15.50	40	pendiente = 0
API autenticada	6	Ibuprofeno 800 mg Forte	S/ 15.50	40	respuesta 200

4.3 Pruebas automatizadas

La ejecución de **:shared:testAndroidHostTest** terminó con **54 pruebas aprobadas y 0 fallidas**. Siete pruebas de **ProductoRepositorioOfflineTest** usan SQLite en memoria y Ktor MockEngine; incluyen el caso concurrente **registrarMientrasSincroniza_noPublicaDosVecesElMismoPendiente**, que falló antes de proteger registrar con el Mutex y pasó después. Tres pruebas nuevas de **ProductoOfflineViewModelTest** residen en commonTest y verifican lectura del catálogo sin red, alta pendiente y reconciliación al reconectar mediante un repositorio falso.

La captura anterior de Gradle con 50/50 pruebas corresponde a una ejecución histórica. El resultado actual de 54/54 se comunica como salida de la ejecución local; no se reutiliza esa captura como si fuese nueva.

5. Análisis y conclusiones

Aspecto	Antes: REST directo	Ahora: Offline-First
Lectura	Dependía de la respuesta HTTP.	Proviene del Flow observado en SQLite.
Sin red	No se podía actualizar el catálogo.	El catálogo ya persistido sigue visible.
Alta	Requería servidor disponible.	Se guarda con ID negativo y marca pendiente.
Reconciliación	No aplicaba.	Se reemplaza el ID dentro de una transacción local.

La actividad demuestra la disponibilidad local de productos y la continuidad de altas en modo avión. Los IDs negativos separan el estado local del remoto; el Mutex compartido evita duplicar el POST cuando coinciden un registro y una sincronización. Las pruebas de repositorio y ViewModel cubren los flujos centrales, mientras que la respuesta autenticada del backend confirma la presencia del producto sincronizado.

Límites y trabajo pendiente

La transacción SQLite es local, no distribuida con el servidor: una caída después de un POST exitoso y antes del reemplazo local podría requerir un mecanismo de idempotencia para evitar duplicados al reintentar. La hora de última sincronización se mantiene en memoria y no debe describirse como persistente tras reinicio. Para una entrega con evidencia visual literal de la guía aún conviene obtener una captura auténtica de Android Studio Database Inspector y otra de la respuesta autenticada en Swagger UI. La implementación y pruebas de iOS quedan fuera del alcance acordado con el docente.

Referencias de implementación

Guía práctica y actividad autónoma de la sesión 11, Desarrollo de Aplicaciones Móviles, Universidad Peruana Unión; código fuente de PharmaMobil en los módulos shared y androidApp; documentación oficial de SQLDelight 2.1.0 (Multiplatform SQLite y consultas reactivas).